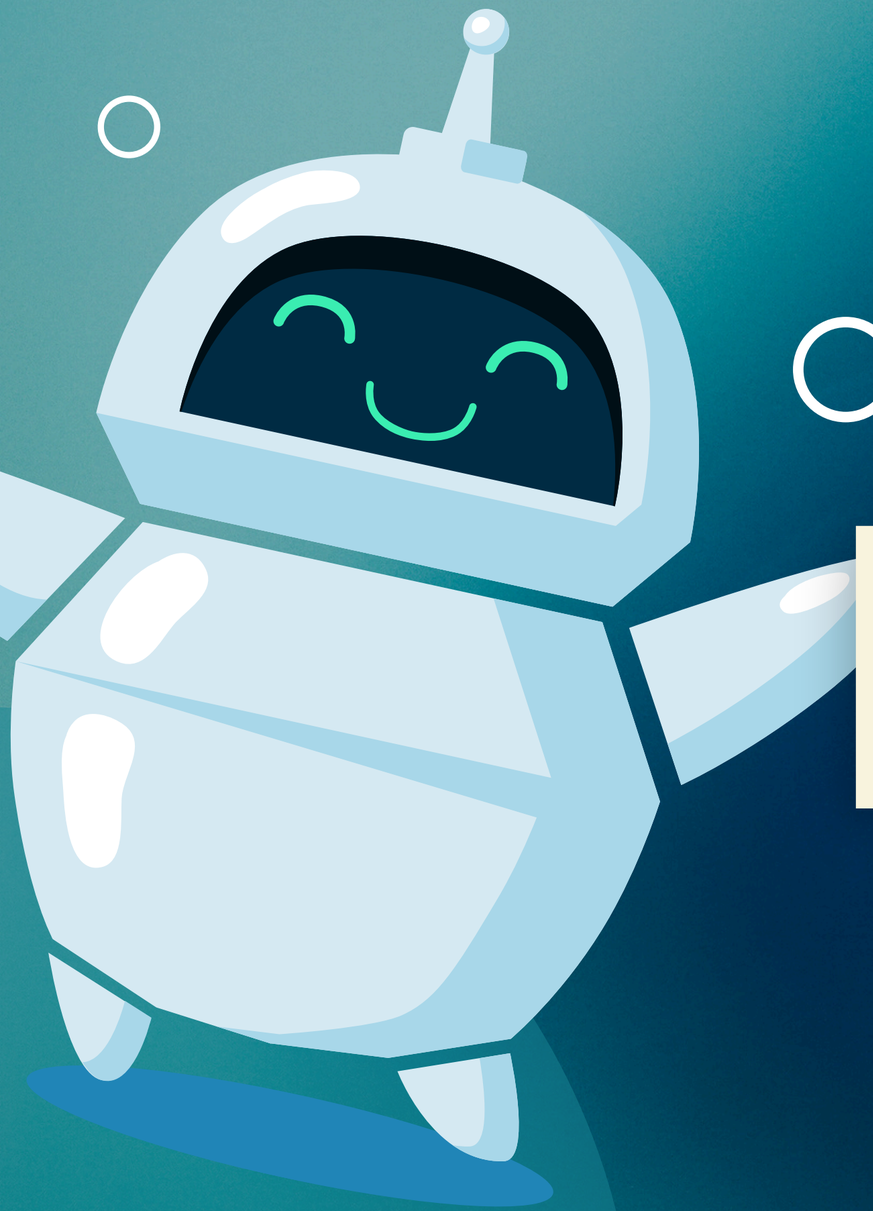


FACYT

Presented by
Brandon Zanotti
Michelle Sayas

RAID Array Simulator

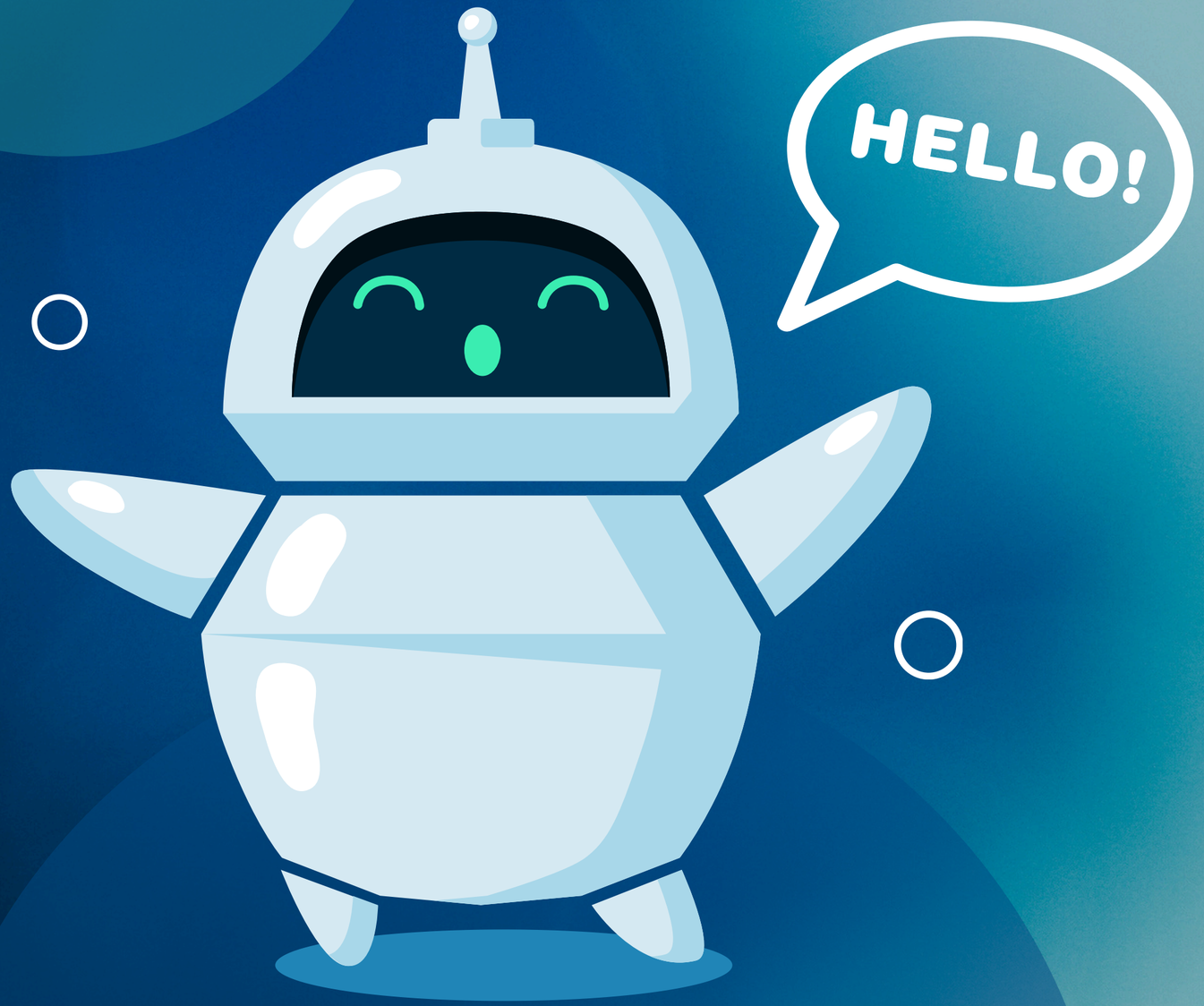
RAID 0, 1, 5, 10

Introducción

El proyecto consiste en el diseño e implementación de un simulador interactivo en C++ que modela el comportamiento de cuatro arquitecturas RAID (0, 1, 5 y 10). El objetivo principal es resolver el conflicto entre rendimiento de lectura/escritura, aprovechamiento de capacidad y tolerancia a fallos en el almacenamiento masivo.

Mediante una arquitectura modular orientada a objetos, el simulador permite:

- Evaluar arquitecturas
- Simular escenarios reales
- Comparar métricas





Problema fundamental

El proyecto se centra en tres aspectos principales:

EL DILEMA DEL ALMACENAMIENTO FÍSICO:

Las unidades de disco duro individuales representan puntos únicos de fallo que limitan la velocidad de acceso (E/S) y ponen en riesgo la disponibilidad de la información ante fallos de hardware.

EL COMPROMISO ENTRE MÉTRICAS:

Al diseñar un sistema de almacenamiento masivo, existe un conflicto inherente entre maximizar la velocidad (rendimiento), aprovechar al máximo el espacio útil y garantizador la tolerancia a fallos (redundancia).

LA NECESIDAD DE ABSTRACCIÓN:

El reto técnico radica en diseñar un simulador en C++ que abstraiga el funcionamiento atómico de la memoria secundaria para gestionar lógicamente múltiples discos físicos, permitiendo modelar el cálculo de paridad (XOR), inyectar fallos controlados y reconstruir datos en tiempo real.

Soluciones



- Diseño Orientado a Objetos: Modela el sistema mediante encapsulamiento, herencia y polimorfismo en C++. Usa Bloque como unidad contenedora de datos/paridad, Disco para gestionar el estado físico y la memoria, y RAIDSystem como clase base abstracta.
- Gestión de Memoria: Aplicar punteros inteligentes para instanciar dinámicamente cada nivel RAID sin fugas de memoria.
- RAID 0: Distribuye datos por módulo sin redundancia.
- RAID 1: Duplica caracteres en 2 discos y recupera mediante copia directa.
- RAID 5: Distribuye datos y paridad con rotación inversa; usa la función lógica XOR para lecturas degradadas y reconstrucción al vuelo.
- RAID 10: Aplica distribución sobre parejas en espejo; reconstruye clonando el disco par de la misma pareja.



Modelado del Disco

- Modela el comportamiento y estado operativo de un disco duro físico individual.
- Encapsula unidentificador entero int id, un estado booleano bool activo y uncontenedor vectorial std::vector<Bloque> bloques que actúa como la memoria secundaria simulada.
- Incorpora el método fallar(), el cual invalida el disco (activo = false) y destruye el contenido de la memoria ejecutando bloques.clear().
- Define el método reemplazarYReconstruir(size t numBloques), que restablece la unidad (activo = true) y reinicializa el vector con bloques vacíos preparados para recibir la reconstrucción.

RAID

RAID 0 (STRIPING / DISTRIBUCIÓN POR BLOQUES):

- Divide y distribuye secuencialmente la información entre todos los discos mediante la operación módulo.
- Ofrece el máximo rendimiento de lectura/escritura y un 100% de capacidad útil. Sin embargo, carece totalmente de redundancia: si falla un solo disco, todo el arreglo se inactiva y los datos se pierden de forma irreversible.

RAID 1 (MIRRORING / ESPEJEO):

- Mantiene una copia idéntica de la información en dos unidades físicas en paralelo.
- Prioriza la alta disponibilidad y la simplicidad en la recuperación (copia limpia direct de vector a vector). Su desventaja es la ineficiencia espacial, ya que penaliza la capacidad útil al 50% del total instalado.

RAID 5 (PARIDAD DISTRIBUIDA):

- Distribuye los datos y bloques de control de paridad a lo largo de un mínimo de 3 discos, aplicando una rotación inversa del disco de paridad por franja.
- Optimiza el aprovechamiento del espacio. Soporta la caída de hasta 1 disco recalculando la información en lecturas degradadas o reconstrucciones mediante operaciones lógicas a nivel de bits (XOR). Si caen 2 discos, se bloquea el sistema.

RAID 10 (ARREGLO HÍBRIDO STRIPING + MIRRORING):

- Combina el rendimiento de distribución del RAID 0 con la replicación por parejas del RAID 1 (requiere un número par de discos, mínimo 4).
- Presenta una capacidad útil fija del 50%. Ofrece alta tolerancia a fallos descentralizada: puede soportar la caída de múltiples discos siempre que no fallen simultáneamente los dos integrantes de la misma pareja de espejos.



TABLA COMPARATIVA

M´etrica / Nivel RAID	RAID 0	RAID 1	RAID 5	RAID 10
Capacidad util	100% (NC)	50%(1C)	N−1 N 100%	50%(N 2 C)
Discos mínimos	2 unidades	2 unidades	3 unidades	4 unidades (pares)
Tolerancia máxima fallos	0 discos	1 disco	1 disco	Hasta 1 disco por parejas
Mecanismo redundante	Ninguno	Espejeo Directo (1:1)	C´alculo Paridad XOR (⊕)	Espejeo en Parejas Distribuidas

Conclusión del Analisis



BALANCE COSTO/BENEFICIO (RAID 5):

Es la solución matemáticamente más eficiente para optimizar recursos. Reduce la pérdida de espacio útil a la capacidad de un solo disco, asumiendo a cambio un costo computacional adicional en operaciones de bits (XOR) durante la reconstrucción y las lecturas en modo degradado.

REDUNDANCIA CRÍTICA VS. DESEMPEÑO (RAID 10):

Resultó ser superior a RAID 1 en escalabilidad y mejor que RAID 5 en simplicidad de recuperación, ya que recupera datos mediante copia directa sin requerir cálculos al vuelo. Es la mejor alternativa para entornos críticos con altas exigencias de rendimiento y disponibilidad.

VALIDACIÓN DEL SOFTWARE:

Se comprobó que las abstracciones orientadas a objetos desarrolladas en C++ (Bloque, Disco y RAIDSystem) representan fielmente las reglas de la virtualización de almacenamiento físico, permitiendo auditar y verificar el comportamiento determinista de cada algoritmo al inyectar fallos en el sistema.